



Réunion 1

Clément Sinon

Table des matières

1	Introduction	3
1.1	Motivation du plan	3
1.2	Motivation de la réunion	3
1.3	Répartition des tâches	3
1.4	Echauffement	4
1.5	Extreme programming	4
2	Structure du groupe	5
2.1	Prise de décisions	5
2.2	Hiérarchie en arborescence ou platte	5
2.3	Répartition des tâches	6
2.4	Organisation des réunions	6
2.5	Traces écrites	7
3	Programmation	8
3.1	Pair programming	8
3.2	Conventions de code	8
3.3	Dépôt GitLab	9
3.4	User stories	9
3.5	Début de l'itération 1	9
4	Divers	11
4.1	Serveur Discord	11
4.2	Non prévu	11

Chapitre 1

Introduction

1.1 Motivation du plan

Ce document présente une proposition de plan pour la première réunion de notre projet du cours de Génie Logiciel et Gestion de Projet. L'ordre des sections correspond à l'ordre dans lequel les sujets seront abordés lors de la réunion.

Trois objectifs motivent l'existence de ce document. Par ordre décroissant d'importance, ceux-ci sont les suivants :

Clarifier les idées

Mettre à l'écrit des idées abstraites ou peu familières permet de les clarifier pour la personne qui les formule.

Donner une direction

Afin que la réunion ne s'éternise pas et ne gaspille pas inutilement le temps des participants, il est essentiel de lui donner une direction. Il est donc nécessaire de définir en premier la direction en question.

Annoncer les votes

Il est dur de donner et défendre son avis quand on est pris au vif. Débattre sans avoir réfléchi au préalable peut aussi aboutir à des discussions inutilement longues. Les décisions qu'il sera demandé de prendre seront donc annoncées dans ce document afin de laisser à chacun le temps de s'y préparer.

1.2 Motivation de la réunion

La réunion a pour but primaire de définir une structure pour notre groupe. Si le temps nous le permet, nous aborderons aussi l'organisation du travail de programmation.

Le but principal est de finir le chapitre portant sur la structure du groupe. Les décisions portant sur les aspects de programmation peuvent être laissés à une autre réunion.

1.3 Répartition des tâches

Pour le bon déroulement de la réunion, plusieurs tâches seront attribuées.

Tâche 1.1 Chef d'orchestre

Annonce et motive les sujets de discussion.

Tâche 1.2 Maître du temps

Estime le temps que chaque décision prendre et avertit lorsque le temps menace d'être dépassé. Empêche les discussions de tourner en rond ou de perdre le fil.

D'autres tâches non relatives au déroulement de la réunion seront proposées au fil de celle-ci.

Tâche 1.3 Bookkeeper

Note diverses informations relatives à la réunion :

- présences et retards,
- décisions prises,
- résumé.

Tâche 1.4 Arbitre

Déclare les votes. Déclare le début et la fin des débats. S'occupe du comptage des votes.

Tâche 1.5 Tie-breaker

En cas de vote 50/50, sa décision prime. Sera discuté en début de réunion.

1.4 Echauffement

La réunion débutera par un tour de table. Chaque personne présentera tour à tour :

- ce qu'elle souhaite apprendre,
- ce qui lui donne des difficultés dans les projets,
- ses attentes.

Cette activité a trois objectifs : définir le ton de la réunion, anticiper les points de friction et verbaliser les différences entre les membres du groupe. Le ton espéré pour la réunion est celui d'une discussion calme où chacun peut s'exprimer à son rythme sans se faire interrompre. Il est donc impératif de faire respecter ce ton durant cet échauffement.

1.5 Extreme programming

Il nous est demandé dans le cadre de ce projet de pratiquer l'*extreme programming* (XP). Cette pratique est très étrangère à ce que nous avons pratiqué jusqu'à présent.

XP pousse les bonnes pratiques à l'extrême :

- les tests unitaires c'est bien → on fait des tests unitaires avant de développer,
- le *code review* c'est bien → on fait du *code review* h24 (*pair programming*),
- ...

Les principes clefs à pratiquer sont :

Test-driven development (TDD)

Ecrire des tests unitaires avant de coder. Ne coder que le strict nécessaire pour valider les tests. Pratiquer le cycle "red → green → refactor".

Collective ownership

Il n'y a pas de séparation des tâches entre les développeurs. Aucun code "n'appartient" à une personne. Tout le monde est incité à refactoriser le code des autres.

Pair programming

Le *code review* est constant et prend lieu pendant le moment même de la programmation. Les développeurs travaillent en binômes : l'un est le pilote et code, l'autre est le copilote et révise ce que le pilote fait.

Continuous integration

On intègre constamment son code avec celui des autres. Les merges fréquents sur *main* sont encouragés.

La philosophie d'XP prône :

- du code court et simple,
- qualité > quantité,
- des conventions de code rigoureuses,
- du travail régulier (pas de *bursts* ni de nuits blanches).

Chapitre 2

Structure du groupe

2.1 Prise de décisions

Espérer un vote unanime pour chaque décision dans un groupe de 10 serait irréaliste. Passer par un vote majoritaire à chaque fois serait vite pénible pour certains membres. Débattre de chaque décision serait encore pire sur ce point.

Par ailleurs, les débats en ligne sont longs. Il faut se mettre d'accord sur les options avant de démarrer le vote, ce qui requiert un débat. Certains mettent plusieurs heures, si pas jours, avant de répondre. Il est dur de distinguer les membres non intéressés par le débat de ceux qui n'ont pas encore regardé leurs messages.

Nous souhaitons donc établir un système de prise de décision qui soit **léger** (peu de membres doivent être dérangés) et **rapide**. Une approche classique est de traiter les décisions différemment en fonction de leur impact :

Petit impact

Les décisions à faible impact sont prises par les membres à qui ont été confiée la tâche concernée. Ils n'ont pas d'obligation de consulter les autres membres du groupe. Si quelqu'un exprime un désaccord sur le caractère à "petit impact" de la décision, celui-ci sera discuté lors de la prochaine réunion.

Large impact

Si une décision impacte le groupe dans son ensemble, elle est prise lors de la prochaine réunion. Si la décision ne peut pas attendre, elle est d'abord prise sans consultation par les membres en charge puis sera discutée et votée proprement à la prochaine réunion.

Vote 2.1

Voter sur une méthode de prise de décision telle que décrite ci-dessus.

2.2 Hiérarchie en arborescence ou platte

Il est naturel d'organiser un groupe en une arborescence de sous-groupes. Des délégués peuvent représenter leurs coéquipiers lors des réunions, permettant une plus facile organisation de celles-ci. Les *user stories* peuvent être groupées en modules pouvant être assignés à des équipes.

Cependant, l'*extreme programming* prône une hiérarchie platte et rejette toute séparation des membres et du code. Les développeurs s'apparient en binômes qui sont fréquemment changés afin de permettre une meilleure diffusion de l'information. Les *stories* ne sont pas assignées à l'avance. Il est à noter que le cours pousse à l'adhérence à XP.

Vote 2.2

Déterminer à quel point embracer l'*extreme programming*.

2.3 Répartition des tâches

Les membres peuvent vite être surchargés si un investissement en temps ou trop de responsabilités leur sont demandés. Le groupe peut aussi paraître régit par une ou deux personnes si celles-ci investissent trop de temps ou encaissent trop de responsabilités par rapport aux autres. Il pourrait donc être préférable de séparer les différentes tâches et responsabilités en autant de petits rôles unitaires que possible et de les distribuer de façon égale aux différents membres. Tous les membres ne présentant pas autant d'initiative les uns que les autres, il peut aussi être nécessaire de clairement assigner les rôles.

L'assignation volontaire aux tâches est préférable quand possible et équitable, mais ne l'est que rarement. Quelqu'en soient leurs raisons, certains membres ne se désigneront pas volontaires pour beaucoup de tâches. Par ailleurs, certaines tâches que personne ne souhaite faire seront délaissées.

Un système qui vise à permettre à la fois un fonctionnement démocratique et organique, ainsi qu'un fonctionnement efficace et équitable pourrait être d'officialiser certaines tâches mais pas toutes :

Tâches organiques

Peuvent être créées par n'importe quel membre, sans besoin d'approbation des autres. Assignées sur base volontaire. Peuvent être officialisées lors d'une réunion si délaissée mais jugée nécessaire.

Tâches officielles

Décidées lors d'une réunion. Si personne ne se désigne volontaire après un certain délai, quelqu'un est désigné volontaire.

Vote 2.3

Se décider sur un système de création et d'assignation de tâches.

Vote 2.4

Canal Discord pour la création et l'assignation de tâches.

2.4 Organisation des réunions

Les réunions en personne avec présence unanime sont ardues à organiser. Elles sont dérangeantes pour les membres qui habitent loin du campus. Les réunions en appel vocal quant à elles sont prônes à des problèmes techniques ainsi qu'à des tensions et irritations interpersonnelles.

Les réunions peuvent être planifiées à l'avance à des dates précises. Elles peuvent aussi être planifiées au fil de leur nécessité. Une autre approche est de prévoir la prochaine réunion à la fin de chaque réunion.

La présence unanime à chaque réunion est probablement irréaliste. Il pourrait être utile de déterminer le nombre minimum de participants pour une réunion, sans quoi celle-ci est annulée ou reportée.

Vote 2.5

Se décider sur l'organisation des réunions.

Tâche 2.1 Meeting scheduler

Chargé de décider des dates, heures et fréquences des réunions.

Un plan de réunion est un outil utile pour donner une direction aux réunions. Il est aussi essentiel d'annoncer les votes autant à l'avance que possible afin de permettre à chacun de réfléchir à ses positions.

Vote 2.6

Date limite de publication d'un plan final.

Un moyen de permettre à chaque membre de régulièrement partager son avancée avec le groupe pourrait être implémenté. Cela pourrait se faire en début de chaque réunion. De courtes réunions type [stand-up meetings](#) pourraient être organisées à cet effet.

Vote 2.7

Canal Discord pour poster des propositions de sujets à mentionner aux réunions.

Tâche 2.2 Meeting planner

Prépare un plan des réunions. Les votes doivent être clairement annoncés.

Vote 2.8

Méthode de partage d'avancée.

2.5 Traces écrites

Garder une trace écrite de diverses informations permet de créer une mémoire objective. Ce n'est pas tant une arme administrative à utiliser les uns contre les autres, mais plutôt une façon de clarifier à l'écrit les idées émises ainsi qu'une protection contre le cerveau humain, qui est prône à réinterpréter ses souvenirs.

Il faut cependant faire attention à ne pas faire de *bookkeeping* en excès ; écrire les idées prend du temps et un livre trop dense ne sera pas lu.

Vote 2.9 Informations soumises au bookkeeping

Quelques idées :

- décisions prises,
- tâches et rôles assignées,
- *deadlines*,
- présences et retards en réunions,
- heures prestées,
- résumé des réunions.

Chapitre 3

Programmation

3.1 Pair programming

XP prône le travail en binômes. L'un est le pilote et code, l'autre est le copilote et fait du *code review* en *live*.

Les binômes devraient être changés régulièrement afin de ne pas créer de “silo” de connaissance dans le code. Idéalement, d'ici la fin du projet, tout le monde aura travaillé au moins une fois avec chaque autre membre. Par conséquent, décider des binômes par affinités personnelles peut être jugé contraire à la philosophie d'XP. Une tournante prédéfinie ou un tirage aléatoire pourraient être envisagés.

Une difficulté que nous rencontrerons qui est d'ordinaire inexistante en entreprise est nos différences de disponibilités.

Le cours prévoit 11h de travail par semaine par étudiant. Ceci inclut le travail en tant que pilote et en tant que copilote.

Vote 3.1

Comment décider des binômes.

Certains membres pourraient aussi embrasser un rôle de support. Ils pourraient être contactés en cas de difficulté par un binôme. Ces membres auraient des contraintes de disponibilité différentes des autres.

Vote 3.2

Création d'un rôle dédié au support.

En prévisions de potentiels problèmes de personnel, il pourrait être utile de remplir des fiches de rapport de session. Celles-ci renseigneraient les éventuels retard, un ressenti sur la session ou autre retour.

Vote 3.3

Fiches de rapport de session.

3.2 Conventions de code

La propreté et l'homogénéité du code sont essentielles au bon fonctionnement de l'*extreme programming*. Il est donc impératif de définir des conventions.

Un socle d'XP est le *test-driven development*. Les tests unitaires sont écrits avant les fonctionnalités qu'ils testent. Aucune fonctionnalité n'est implémentée si elle ne sert pas à valider un test.

Vote 3.4 Conventions de code

- *snake case* ou *camel case*,
- anglais ou français,
- noms des *getters*, *setters* et *unit tests*,
- indentation avec espaces ou tabulations,
- complétude de la *javadoc* (@param, @return, @throws, @author, @version),

Vote 3.5

Code-t-on des tests unitaires pour les constructeurs, *getters* et *setters* ?

3.3 Dépôt GitLab

Un dépôt GitLab sera fourni pour le groupe par les assistants du cours.

Suivant la philosophie XP, le code sera constamment manipulé et réfactoré par tous les membres du groupe. Cela mènera inévitablement à du chaos. Afin de lutter contre cela, définir une structure des dossiers et fichiers pourrait s'avérer utile.

Vote 3.6

Structure des dossiers et fichiers sur le dépôt.

Vote 3.7

Format des commits. Précise-t-on les copilotes en tant que co-auteurs ?

3.4 User stories

Le projet est défini par une liste de *user stories* – essentiellement, de fonctionnalités – à implémenter. Une tentative de les organiser visuellement est fournie en [figure 3.1](#).

A priori les *stories* ne sont pas assignées à l'avance en XP. Elles sont réservées par un binôme en début de session de travail, et pour la durée de cette session. Cependant certains membres pourraient avoir des affinités particulières pour une *story* plutôt qu'une autre.

Vote 3.8

Comment attribuer les *stories* ?

Vote 3.9 Outils de gestion de tâches

GitLab offre par défaut un système Kanban pour organiser les tâches à réaliser. D'autres outils tels que des [diagrammes de Gantt](#) pourraient être envisagés.

3.5 Début de l'itération 1

Il est demandé pour le mardi 24 février d'estimer pour chaque *story* :

Son importance

Une priorité donnée à la *story*, de 1 à 3.

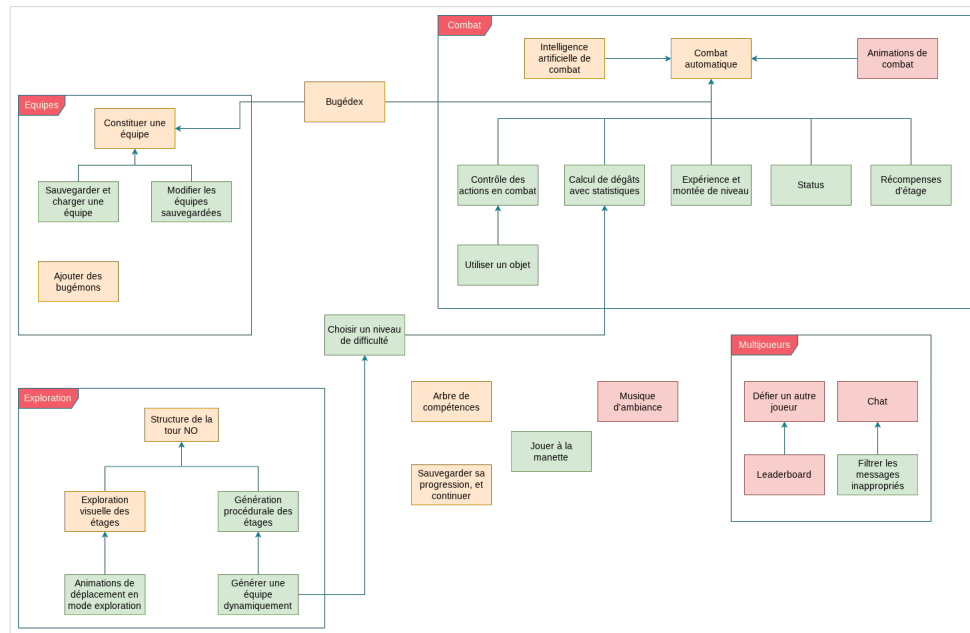
Son risque

La difficulté qu'elle représente, de 1 à 3.

Son temps

Le nombre d'heures de programmation qui devront lui être dédiées.

Figure 3.1 Plan des user stories



Vote 3.10

Comment évaluer les stories. Se les répartir ou faire ça en commun ?

Chapitre 4

Divers

4.1 Serveur Discord

Le serveur Discord est jusqu'à présent la plateforme de communication officielle du groupe. Cette plateforme n'a cependant pas été choisie par vote.

Une courte explication du rôle de chaque canal du serveur pourrait être demandée.

Vote 4.1

Retirer le canal "bienvenues".

Vote 4.2

Ajouter un canal "annonces" obligatoire.

4.2 Non prévu

Les sujets qui n'ont pas été inscrits dans le plan pourraient être discutés en fin de réunion si le temps le permet.

Cependant, permettre des vote pour ceux-ci nuirait à la volonté de laisser les membres préparer leurs positions à l'avance avec le plan de réunion.

Vote 4.3

Permettre les votes ou non sur les sujets non annoncés.